

Laboratorio di Elettronica Digitale

- Introduzione al Linguaggio VHDL -

Prof. Stefano Ricci

INTRODUZIONE

- Il VHDL è un linguaggio standard per la descrizione ad alto livello di circuiti digitali
- Insieme al VERILOG fa parte degli Hardware Description Language (HDL)
- E' stato introdotto negli anni '80 nell'ambito di un progetto del dipartimento della difesa USA denominato Very High Speed Integrated Circuits (VHSIC)
- La sigla "VHDL" abbrevia quindi VHSIC HDL
- Nel 1987 il VHDL è stato adottato come standard dalla IEEE
- Ultima revisione pubblicata è del 2019 (IEEE Std 1076-2019)
- Ultima revisione supportata da Quartus è la 2008 (EEE 1076-2008)
- link interessanti www.vhdl.org

Scopo del linguaggio VHDL è facilitare la progettazione di circuiti e sistemi digitali consentendo la descrizione dell'hardware ad alto livello

CARATTERISTICHE

- Portabilità: Trattandosi di uno standard lo stesso codice VHDL può essere utilizzato su piattaforme di produttori diversi, su ambienti di sviluppo diversi:
 - *Progettazione indipendente dall'HW;*
- Potenza: Con poche righe si possono descrivere strutture di migliaia di gates.

Il lavoro di sintesi è demandato all'ambiente di sviluppo, la realizzazione fisica è decisa dal sintetizzatore.



La qualità del risultato (es. max freq.) dipende anche dal tool impiegato

SIMULAZIONE & SINTESI

- *Simulazione*: Verifica del comportamento di una rete
- *Sintesi*: Passaggio dalla descrizione testuale ad una descrizione di basso livello (netlist)

- I tool impiegati per la simulazione e la sintesi sono distinti
- la stessa descrizione VHDL può essere elaborata sia per la simulazione che per la sintesi

Solo un sottoinsieme del VHDL è sintetizzabile: Il VHDL è un linguaggio complesso che consente, per esempio operazioni su files, definizione di puntatori ecc. che non hanno corrispettivo hardware.



è necessario conoscere quali costrutti sono sintetizzabili, e con quali limitazioni.

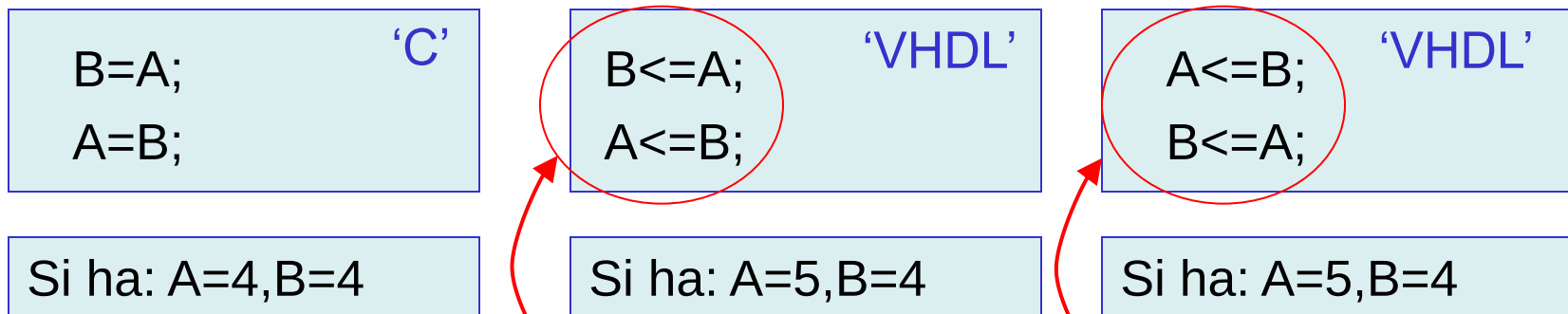


CONCORRENZA

- Concorrenza: A differenza di linguaggi di programmazione “sequenziali” come c/c++, gli HDL consentono di descrivere l'esecuzione contemporanea di più operazioni;

Esempio 1:

Se, partendo con $A=4$, $B=5$ dopo l'esecuzione delle istruzioni:

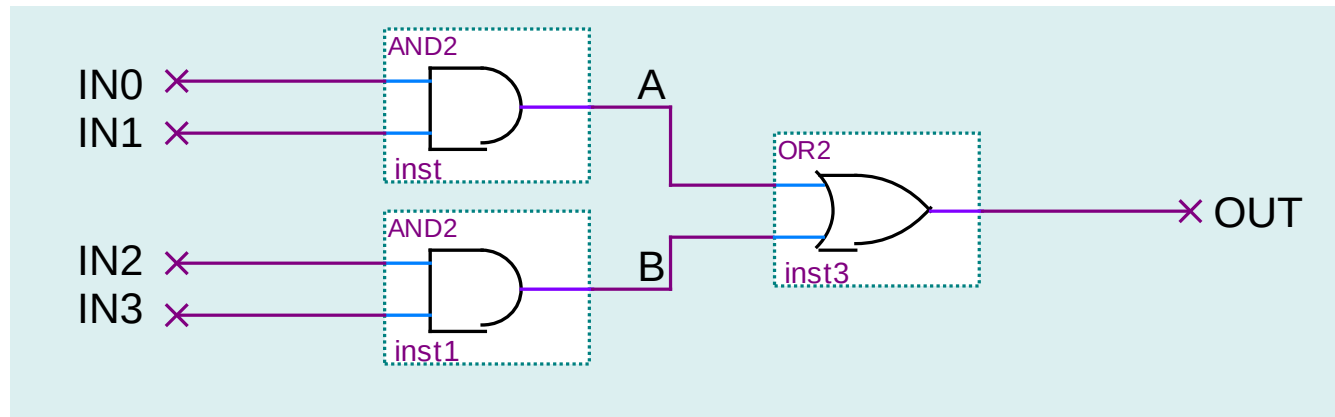


Nota: Poiché l'esecuzione è contemporanea, l'ordine delle istruzioni è ininfluente

CONCORRENZA

Esempio 2:

Data la seguente rete:



Questa può essere descritta in VHDL in vari modi equivalenti, fra cui:

```

A<=IN0 and IN1;
B<=IN2 and IN3;
OUT<= A or B;
  
```

```

OUT<= A or B;
B<=IN2 and IN3;
A<=IN0 and IN1;
  
```

```

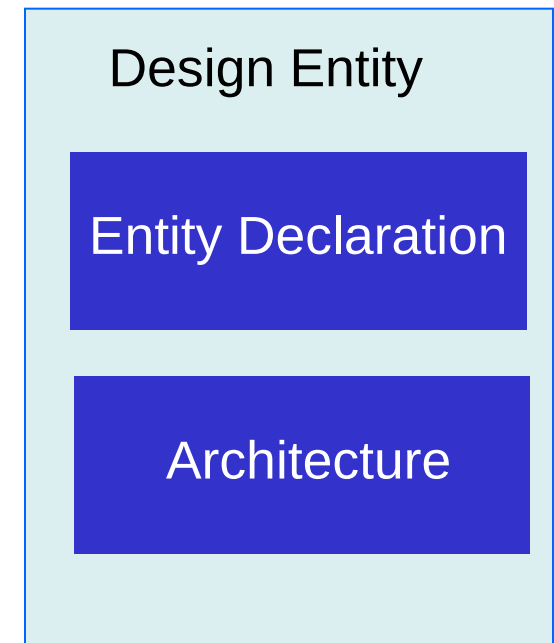
OUT<= (IN2 and IN3) or (IN0 and IN1);
  
```

GENERALITA' VHDL

- I commenti sono preceduti da --
- Il VHDL non è *case sensitive*
- Non è possibile usare spazi nei nomi dei segnali etc
- Non è possibile utilizzare nomi che iniziano o terminano con “_”
- Non è possibile utilizzare nomi che iniziano per numero
- Due “_” consecutivi non sono consentiti
- Non è possibile assegnare parole riservate per i nomi dei segnali etc (es. un segnale denominato BIT non è consentito)
- **E'buona norma usare l'estensione .vhd per i file VHDL**
- Contrariamente al C/C++ non è possibile inserire commenti multi linea (necessario un '--' per ogni linea)
- Le istruzioni terminano con “;”

ENTITY

- Il blocco fondamentale del VHDL è l'*Entity*
- In un certo senso è paragonabile a quello che in 'C++' è un oggetto
- Può rappresentare :
 - Una singola porta logica elementare
 - Un circuito integrato
 - Un intero sistema
 - ...
- E' descritta e contenuta in un file testo con estensione *.vhd e con il nome coincidente con quello dell'*Entity* stessa
- E' composta da:
 - *Entity Declaration*: Descrive l'interfaccia verso il blocco
 - *Architecture*: Descrive come funziona il blocco



ENTITY DECLARATION

```
entity ENTITY_NAME is
  port (PORT_NAME : DIRECTION TYPE;
        ...
        PORT_NAME : DIRECTION TYPE );
end ENTITY_NAME;
```

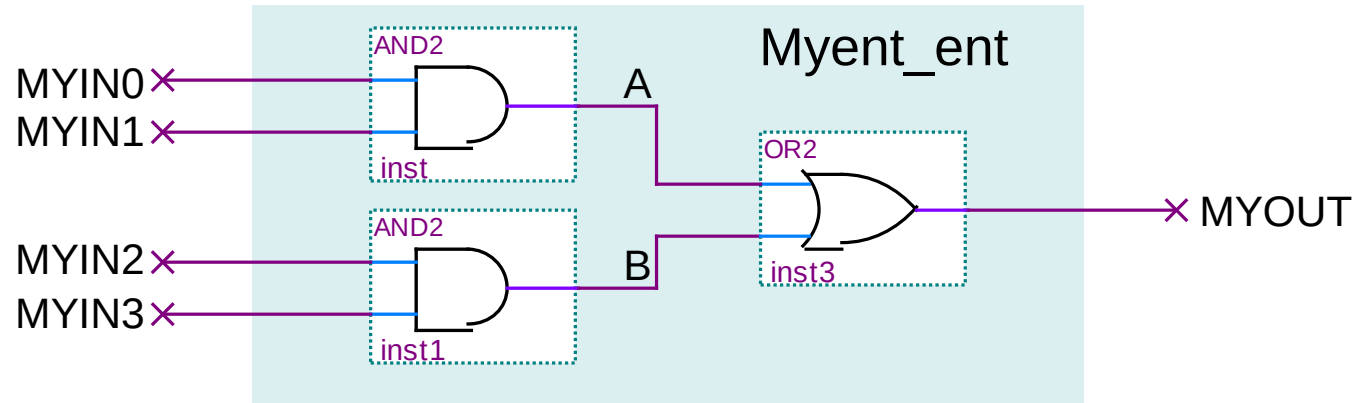
➤ DIRECTION:
in, out, inout

➤ TYPE:
bit, integer,
bit_vector, etc

- L'entity è definita fra le chiavi 'entity' e 'end', ENTITY_NAME è il nome dell'entity e del file che la descrive
- L'interfaccia è definita a seguito della chiave 'port', ciascun segnale scambiato con l'esterno ha un nome (PORT_NAME), una direzione (DIRECTION) e un tipo (TYPE)

ENTITY DECLARATION

Esempio:



```

entity Myent_ent is
  port ( MYIN0 : in bit;
          MYIN1 : in bit;
          MYIN2 : in bit;
          MYIN3 : in bit;
          MYOUT: out bit );
end Myent_ent;
  
```

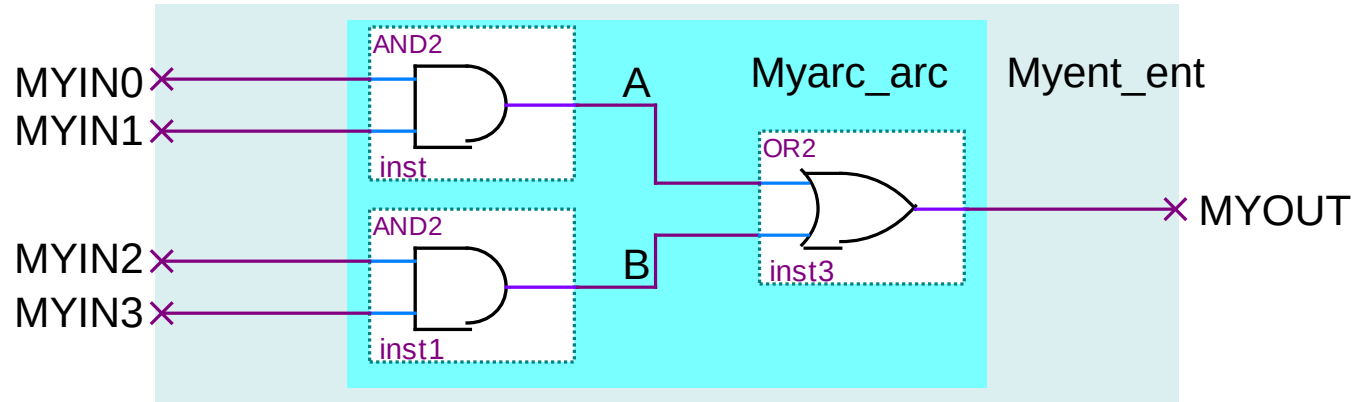
L'entity declaration descrive
l'interfaccia, non il comportamento
del blocco

ARCHITECTURE DECLARATION

```
architecture BODY_NAME of ENTITY_NAME is  
    istruzioni dichiarative  
  
    begin  
        istruzione funzionale  
        istruzione funzionale  
        ...  
    end BODY_NAME;
```

- La dichiarazione si avvale delle parole chiave **architecture**, **begin**, **end**.
- BODY_NAME è il nome di riferimento dell'architettura, ENTITY_NAME è il nome dell'entità a cui si riferisce
- Le *istruzioni dichiarative* permettono di dichiarare variabili, segnali o sottocomponenti che verranno usati nell'architettura
- Le *istruzioni funzionali* permettono di definire il comportamento dell'entity. Queste istruzioni sono **CONCORRENTI**

Esempio:



architecture Myarc_arc **of** Myent_ent **is**

signal A,B: bit;

begin

A <= MYIN0 and MYIN1;

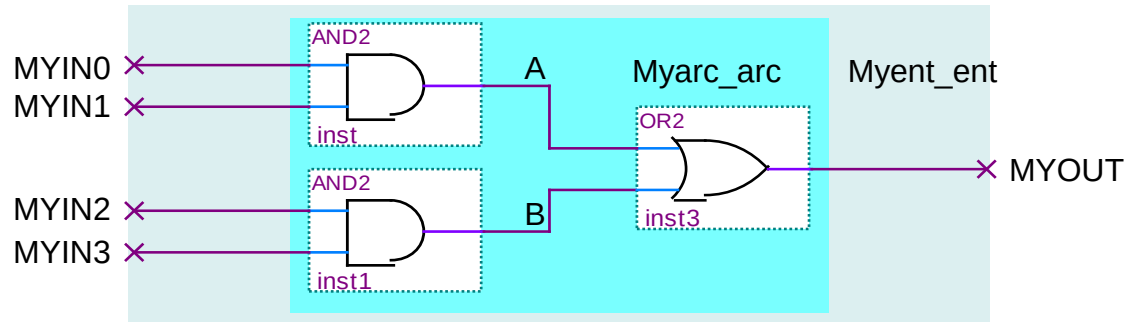
B <= MYIN2 and MYIN3;

MYOUT <= A or B;

end Myarc_arc ;

Dichiarazione dei segnali
“di appoggio” A e B di tipo
bit

ESEMPIO DI ENTITY



```
entity Myent_ent is
  port ( MYIN0, MYIN1, MYIN2, MYIN3 : in bit;
        MYOUT: out bit );
end Myent_ent;
```

Modo più compatto
per dichiarare porte
della stessa
direzione e tipo

```
architecture Myarc_arc of Myent_ent is
  signal A,B: bit;
begin
    A <= MYIN0 and MYIN1;
    B <= MYIN2 and MYIN3;
    MYOUT <= A or B;
end Myarc_arc ;
```

Salvato sul file
'Myent_ent.vhd'

DATI PREDEFINITI

- A ciascun 'dato' è assegnato un tipo, per adesso abbiamo visto 'bit'
- Fra i tipi predefiniti in VHDL vi sono:

NOME	Valori	ESEMPIO
bit	0,1	signal a: bit := '1';
boolean	TRUE, FALSE	signal a: boolean := 'TRUE';
integer	1,2,3,..	signal a: integer range 0 to 10;



- Ma anche:

NOME	RANGE	ESEMPIO
character	'a', 'b', 'c',...	signal a: character := 'a';
time	1ps, 3ns, 4s, etc	signal a: time := 3 ps;
real	1.23, 43.43, etc	signal a: real := 3.14;

Unità fisica
(ps, ns, us,
ms, s)

- A differenza dei primi, questi ultimi NON sono sintetizzabili e si usano nei testbench

DATI PREDEFINITI

➤ E' possibile definire vettori di bit con 'bit_vector'

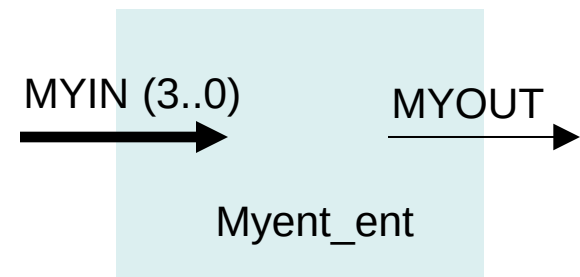
➤ Esempio:

```
signal b: bit_vector (7 downto 0) ;
```

dichiara il vettore 'b' di 8 bit

➤ L'interfaccia dell'entity Myent_ent può quindi essere descritta con l'uso dei vettori:

```
entity Myent_ent is  
  port ( MYIN : in bit_vector (3 downto 0);  
        MYOUT: out bit );  
end Myent_ent;
```



DATI PREDEFINITI

i bit singoli si indicano fra apici,
i gruppi di bit fra doppi apici

```
signal v: bit_vector(3 downto 0) := "1111";
signal p: bit_vector(3 downto 0) := "0000";
```

inizializzazione

- Si accede agli elementi di un vettore mediante indici:

Es. $v(2) \leq p(1);$ v

1	0	1	1
---	---	---	---

Es. $v(3) \leq '0';$ v

0	1	1	1
---	---	---	---

- Si può assegnare una sequenza di bit o un vettore delle stesse dimensioni:

Es. $v \leq "0110";$ v

0	1	1	0
---	---	---	---

 Es. $p \leq v;$ p

1	1	1	1
---	---	---	---

- Si può accedere a specifici set di bit del vettore:

Es. $v(2 \text{ downto } 1) \leq "01";$ v

1	0	1	1
---	---	---	---

Es. $p(1 \text{ downto } 0) \leq v(2 \text{ downto } 1);$ p

0	0	1	1
---	---	---	---

- Si possono concatenare sequenze:

Es. $v \leq "01" \& p(2 \text{ downto } 1);$ v

0	1	0	0
---	---	---	---

L'operatore '&' indica concatenazione, l'operatore 'and' indica 'and logico'

OPERATORI PREDEFINITI

- Sono predefiniti gli operatori 'duali':
and, or, xor, xnor, nor, nand
- E l'operatore 'unario' **not**
- si applicano a dati tipo 'boolean' e 'bit' ('bit_vector')
- il not è prioritario, i rimanenti hanno la stessa priorità
- Se applicati a 'bit_vector' questi devono avere lo stesso numero di bit
- Esempi:
 $a \leq b \text{ and } c \text{ or } d$; (dove a,b,c,d sono bit o bit_vector equivalenti)
 equivale a: $a \leq ((b \text{ and } c) \text{ or } d)$;
 $a \leq b \text{ and not } c \text{ or } d$; equivale a: $a \leq (((b \text{ and } (\text{not } c)) \text{ or } d)$;

OPERATORI PREDEFINITI

- Sono predefiniti gli operatori 'duali' che operano sui dati 'real':

$+$, $-$, $*$, $/$, $**$, etc

Elevazione a potenza



- Sono predefiniti gli operatori 'duali' che operano sui dati 'integer':

$+$, $-$, $*$, etc

Se a, b sono `bit_vector` a 3 bit: non ha senso:

$a + b$;

$a - '001'$;

$a + 1$;

PROCESS

```
entity X_ent is
  port ( ... );
end X_ent;
architecture X_arc of X_ent is
  declarations...
begin
  ....
  process 1
  process 2
  process 3
  ...
end Myarc_arc ;
```

- I processi sono concorrenti (paralleli)
- Le assegnazioni sono casi particolari di processo.
Es. $A \leq B$ **and** C ;
- La sintassi del processo è:

```
My_pr: process ( sensitivity list )
  declarations...
begin
  istruzioni sequenziali...
  istruzioni sequenziali...
end process My_pr;
```

Gli oggetti dichiarati a livello di architettura sono visibili in tutta l'architettura, quelli dichiarati a livello di processo sono visibili nel singolo processo. I signal si possono dichiarare solo a livello di architettura.

PROCESS

```
My_pr: process ( sensitivity list )  
    declarations...  
    begin  
        istruzioni sequenziali...  
        istruzioni sequenziali...  
end process My_pr;
```

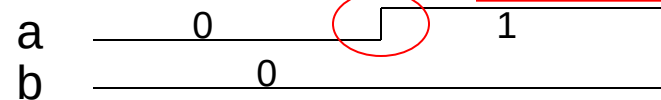
- La “*sensitivity list*” rappresenta la lista dei segnali la cui variazione determina l'*attivazione* del processo
- Quando il processo è *attivo* le istruzioni vengono valutate sequenzialmente (dall'alto in basso) una dopo l'altra.
- La valutazione dell'espressione produce un valore che NON viene immediatamente assegnato alla rispettiva variabile o segnale
- Una volta valutate TUTTE le istruzioni i.e. sono in fondo alla lista, i segnali e le variabili vengono assegnate tutte insieme
- Il processo torna inattivo, in attesa della variazione di un segnale elencato nella *sensitivity list*

PROCESS

Esempio

➤ Il processo è normalmente latente e si attiva quando vi è una variazione *‘evento’* di a o b

Esempio:



```
...
    signal a,b,c,s: std_logic;
...
My_pr: process ( a,b )
    begin
        s <= a;
        c <= a;
        s <= a and b;
    end process My_pr;
```

tempo

- il processo è latente
- a passa da ‘0’ a ‘1’: si attiva il processo
- ad s verrà poi assegnato ‘1’ (a)
- a c verrà poi assegnato ‘1’ (a)
- ad s verrà poi assegnato ‘0’ (1 and ‘0’), questo sovrascrive l’assegnazione precedente
- sono finite le istruzioni, si eseguono le assegnazioni schedate (s diviene ‘0’, c diviene ‘1’)
- il processo torna latente fino al px evento su a o b

Se un segnale ha più assegnazioni schedate, ha effetto solo l’ultima

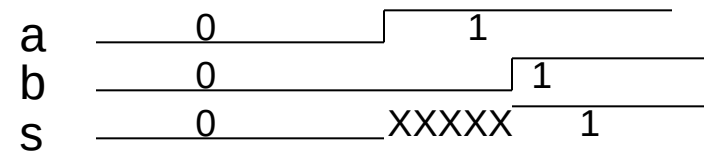
PROCESS

Esempio: stesse assegnazioni fuori e dentro un processo

Concorrenti

```
Architecture My_arc of My_Ent is
  signal a,b,s : std_logic;
begin
  s <= a;
  s <= b;
end My_arc;
```

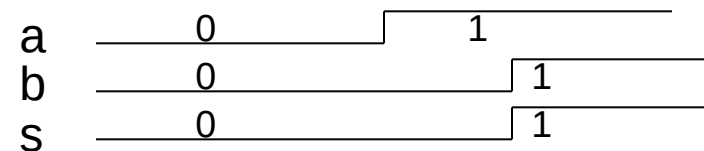
s ha 2 driver. Il sintetizzatore segnala errore e il simulatore assegna ad s il valore 'X', indeterminato in caso di conflitto



Sequenziali

```
...
  signal a,b,s: std_logic;
...
My_pr: process ( a,b )
begin
  s <= a;
  s <= b;
end process My_pr;
```

La seconda assegnazione sovrascrive la prima: s <= a; viene ignorata, vale s <= b;



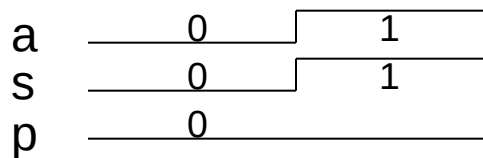
PROCESS

Attenzione: all'interno di un processo le istruzioni sono sequenziali, ma comunque il risultato è diverso da quello che si otterrebbe con la stessa sequenza in 'C'!

Esempio:

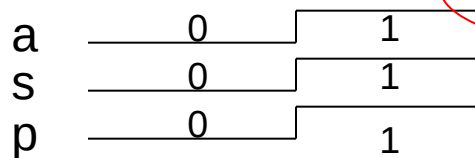
VHDL

```
signal a,p,s: std_logic;
My_pr: process ( a )
begin
    s <= a;
    p <= s;
end process My_pr;
```



'C'

```
int p = func (int a) {
    int static s;
    s = a;
    p = s;
}
```



equivalente 'c' del VHDL

```
int p = func (int a) {
    int static s, s_new;
    s_new = a;
    p_new = s;
    p = p_new;
    s = s_new; }
```

Aggiornamento
effettivo

- In 'c' l'assegnazione di una variabile ha effetto IMMEDIATO;
- In VHDL l'assegnazione di un segnale è schedulata, e sarà effettivamente eseguita solo ALLA FINE del processo.

COSTRUTTI SEQUENZIALI E CONCORRENTI

➤ I costrutti sequenziali e concorrenti si devono usare rispettivamente all'interno e all'esterno del processo

```
entity X_ent is
  port ( ... );
end X_ent;
architecture X_arc of X_ent is
  declarations...
begin
  costrutti concorrenti

  my_prc: process ()
  begin
    costrutti sequenziali

  end process my_proc;

end Myarc_arc ;
```

Sequenziali:

- if – then – elsif - else – end if
- case is – when – end case
- for – loop – end loop
- wait

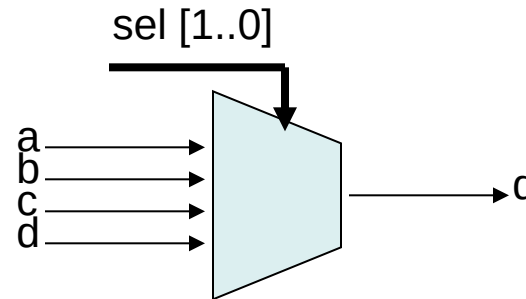
Concorrenti:

- <= - when - else
- whit select - when

L 'assegnazione (<=) si usa sia fuori che dentro i processi

COSTRUTTI CONCORRENTI

Esempio: un MUX 4:1



when - else

```
entity Mux_ent is
  port ( a,b,c,d: in bit;
        sel: in bit_vector(1 downto 0);
        q: out bit);
end Mux_ent;
architecture Mux_arc of Mux_ent is
begin
  q <= a when sel = "00" else
        b when sel = "01" else
        c when sel = "10" else
        d;
end Muxc_arc ;
```

with select - when

```
entity Mux_ent is
  port ( a,b,c,d: in bit;
        sel: in bit_vector(1 downto 0);
        q: out bit);
end Mux_ent;
architecture Mux_arc of Mux_ent is
begin
  with sel select
    q <= a when "00",
        b when "01",
        c when "10",
        d when others;
end Muxc_arc ;
```

COSTRUTTI SEQUEZIALI

case is – when – end case

```
entity Mux_ent is  
  port ( a,b,c,d: in bit;  
         sel: in bit_vector(1 downto 0);  
         q: out bit);  
end Mux_ent;  
architecture Mux_arc of Mux_ent is  
  begin  
    mux: process (a,b,c,d,sel)  
      begin  
        case sel is  
          when "00" =>  
            q <= a;  
          when "01" =>  
            q <= b;  
          when "10" =>  
            q <= c;  
          when others =>  
            q <= d;  
          end case;  
        end process mux;  
      end Mux_arc ;
```

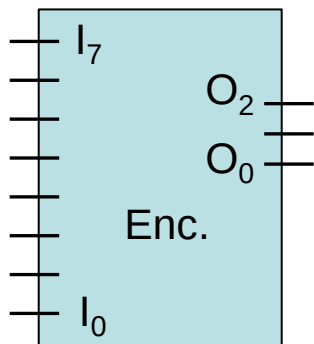
If - then – elsif - else – end if

```
entity Mux_ent is  
  port ( a,b,c,d: in bit;  
         sel: in bit_vector(1 downto 0);  
         q: out bit);  
end Mux_ent;  
architecture Mux_arc of Mux_ent is  
  begin  
    mux: process (a,b,c,d,sel)  
      begin  
        if sel = "00" then  
          q <= a;  
        elsif sel = "01" then  
          q <= b;  
        elsif sel = "10" then  
          q <= c;  
        else  
          q <= d;  
        end if;  
      end process mux;  
    end Mux_arc ;
```

Nota lo spelling!

EXAMPLE : PRIORITY ENCODER

The encoder generates the binary code that corresponds to the active input of higher weight



I ₇	I ₆	I ₅	I ₄	I ₃	I ₂	I ₁	I ₀	O ₂	O ₁	O ₀
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	-	0	0	1
0	0	0	0	0	1	-	-	0	1	0
0	0	0	0	1	-	-	-	0	1	1
0	0	0	1	-	-	-	-	1	0	0
0	0	1	-	-	-	-	-	1	0	1
0	1	-	-	-	-	-	-	1	1	0
1	-	-	-	-	-	-	-	1	1	1

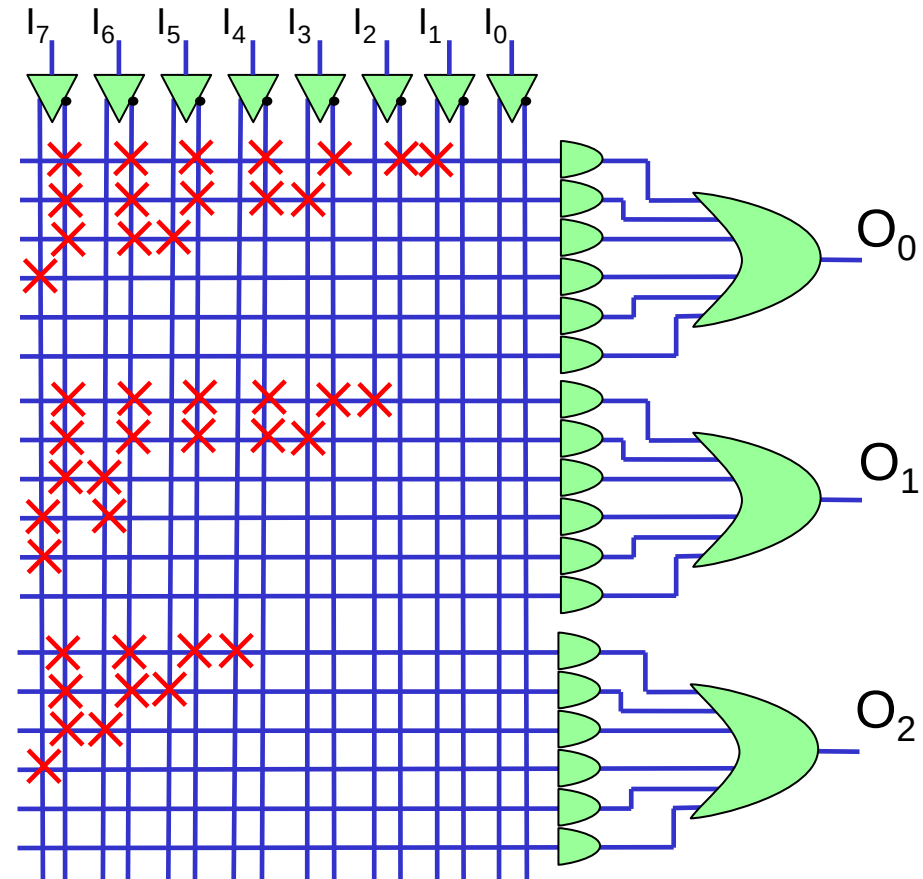
'-' = don't care

← Special case

Encoders are used. e.g. in flash AD converters,

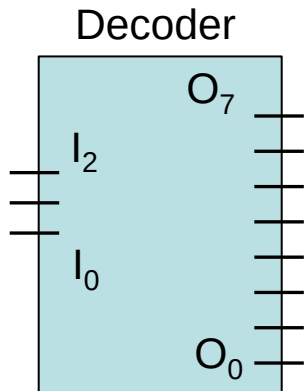
EXAMPLE : PRIORITY ENCODER

```
entity Pencil is
  port ( I: in bit_vector(7 downto 0);
        O: out bit_vector(2 downto 0) );
end Pencil;
architecture Pencil_arc of Pencil is
begin
  Pencil: process (I)
  begin
    if I(7) = '1' then O <= "111";
    elsif I(6) = '1' then O <= "110";
    elsif I(5) = '1' then O <= "101";
    elsif I(4) = '1' then O <= "100";
    elsif I(3) = '1' then O <= "011";
    elsif I(2) = '1' then O <= "010";
    elsif I(1) = '1' then O <= "001";
    else O <= "000";
    end if;
  end process Pencil;
end Pencil_arc;
```



EXAMPLE : DECODER

The decoder generates the output corresponding to the input binary code



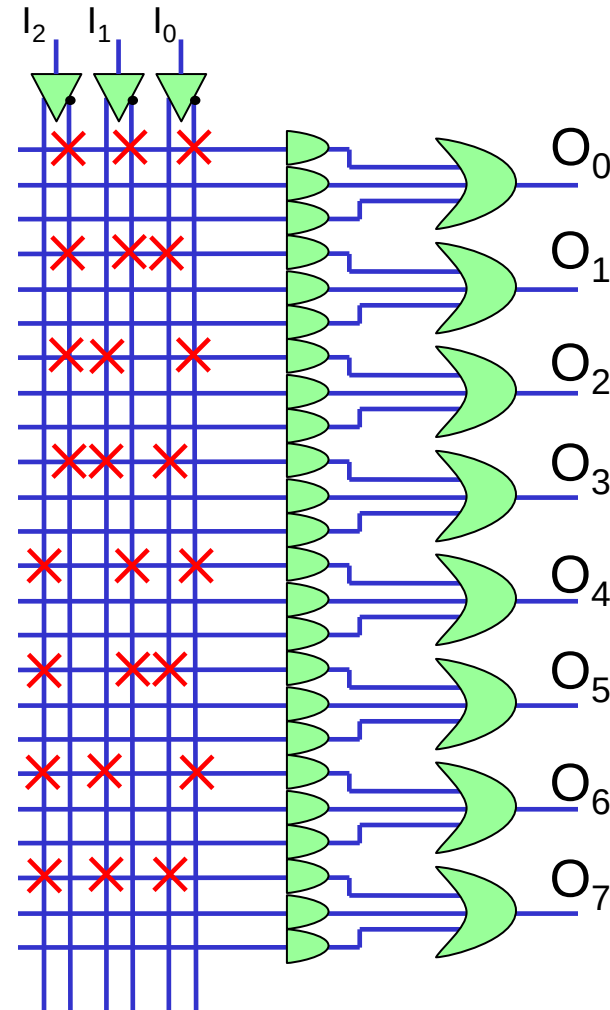
I ₂	I ₁	I ₀	O ₇	O ₆	O ₅	O ₄	O ₃	O ₂	O ₁	O ₀
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

decoders are used. e.g. in RAM for address decoding

EXAMPLE : DECODER

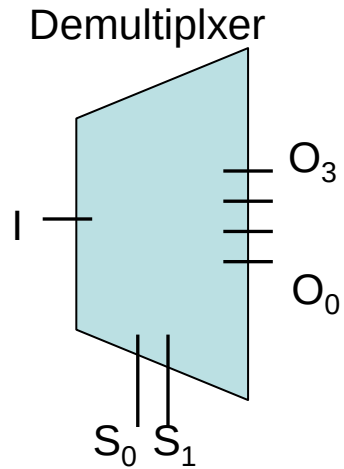
```

entity Dec_ent is
  port ( I: in bit_vector(2 downto 0);
         O: out bit_vector(7 downto 0) );
end Dec_ent ;
architecture Dec_arc of Dec_ent is
  begin
    Dec: process (I)
    begin
      O <= "00000000";
      if I = "000" then O(0) <= '1' ;
      elsif I = "001" then O(1) <= '1';
      elsif I = "010" then O(2) <= '1';
      elsif I = "011" then O(3) <= '1';
      elsif I = "100" then O(4) <= '1';
      elsif I = "101" then O(5) <= '1';
      elsif I = "110" then O(6) <= '1';
      else O(7) <= '1';
      end if;
    end process Dec;
  end Dec_arc ;
  
```



EXAMPLE : DEMULTIPLEXER

The demultiplexer copies the input line(s) to the output line(s) selected by the selection inputs (S). It is similar to the decoder.

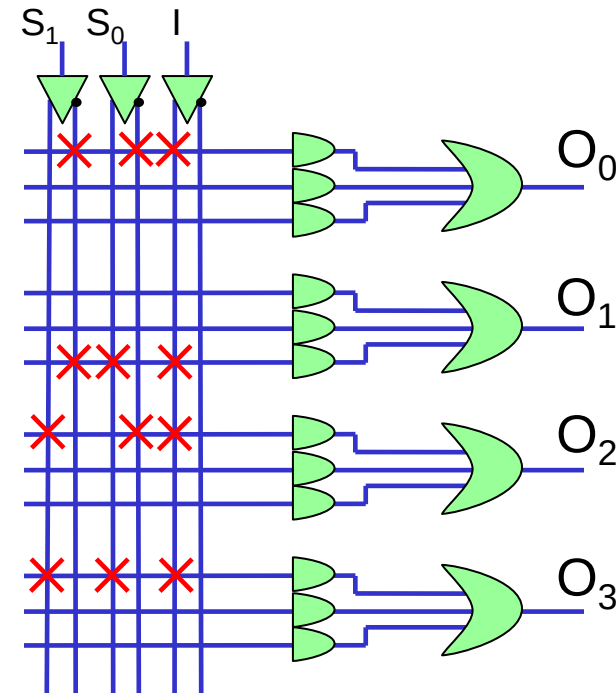


S_1	S_0	I	O_3	O_2	O_1	O_0
0	0	0	0	0	0	0
0	0	1	0	0	0	1
0	1	0	0	0	0	0
0	1	1	0	0	1	0
1	0	0	0	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	0	0
1	1	1	1	0	0	0

EXAMPLE : DEMULTIPLEXER

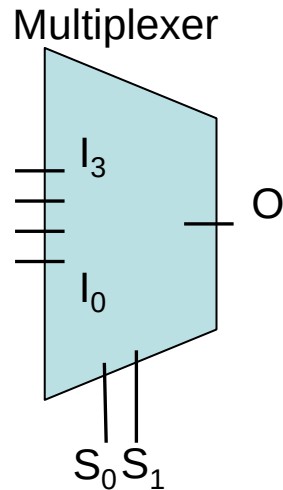
```

entity Demux_ent is
  port ( S: in bit_vector(1 downto 0);
        I: in bit;
        O: out bit_vector(3 downto 0) );
end Demux_ent ;
architecture Demux_arc of Demux_ent is
  begin
    Dem: process (I,S)
    begin
      O <= "0000";
      if S = "00" then O(0) <= I;
      elsif S = "01" then O(1) <= I;
      elsif S = "10" then O(2) <= I;
      else O(3) <= I;
      end if;
    end process Dem;
end Demux_arc ;
  
```



EXAMPLE : MULTIPLEXER

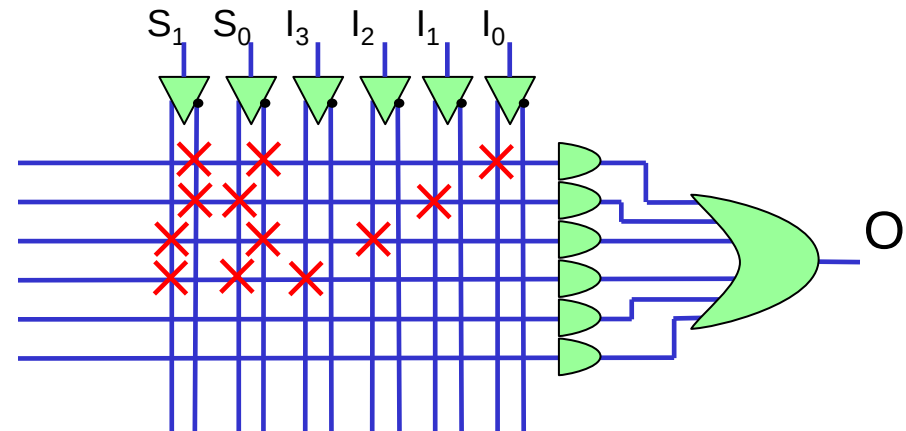
The multiplexer copies the input line(s) to the output line(s) selected by the selection inputs (S). It is similar to the decoder.



S ₁	S ₀	I ₃	I ₂	I ₁	I ₀	O
0	0	-	-	-	0	0
0	0	-	-	-	1	1
0	1	-	-	0	-	0
0	1	-	-	1	-	1
1	0	-	0	-	-	0
1	0	-	1	-	-	1
1	1	0	-	-	-	0
1	1	1	-	-	-	1

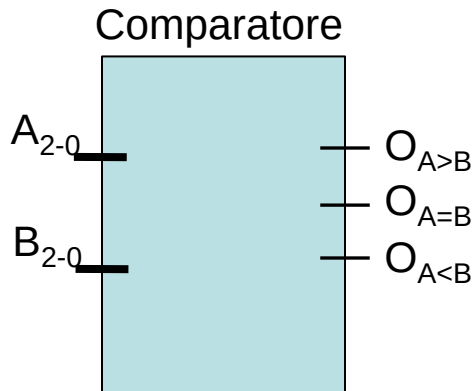
EXAMPLE : MULTIPLEXER

```
entity Mux_ent is
  port ( S: in bit_vector(1 downto 0);
        I: in bit_vector(3 downto 0);
        O: out bit );
end Mux_ent ;
architecture Mux_arc of Mux_ent is
  begin
    Mux: process (I,S)
    begin
      if S = "00" then O <= I(0);
      elsif S = "01" then O <= I(1);
      elsif S = "10" then O <= I(2);
      else O <= I(3);
      end if;
    end process Mux;
  end Mux_arc ;
```



EXAMPLE: COMPARATOR

The comparator compares the A and B inputs and produces 3 bits for the conditions $A=B$; $A>B$; $A<B$. The bit that reports the 'true' condition is '1', the others are '0'.



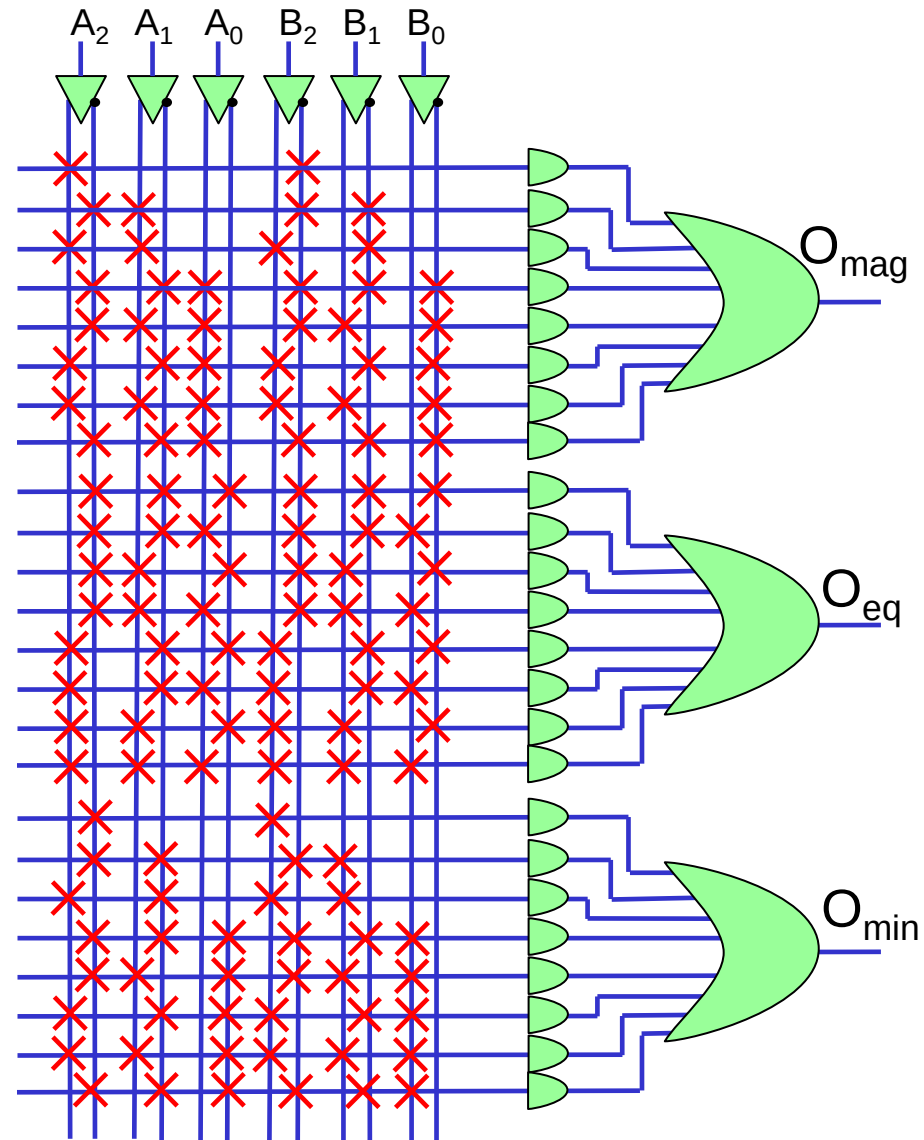
A_2	A_1	A_0	B_2	B_1	B_0	$O_{A>B}$	$O_{A=B}$	$O_{A<B}$
1	-	-	0	-	-	1	0	0
0	1	-	0	0	-	1	0	0
1	1	-	1	0	-	1	0	0
0	0	1	0	0	0	1	0	0
0	1	1	0	1	0	1	0	0
1	0	1	1	0	0	1	0	0
1	1	1	1	1	0	1	0	0
0	0	1	0	0	0	1	0	0
...	...	...	...	...	...	0	...	...

The represented Truth Table is limited to the '1s' of $O_{A>B}$. In fact $O_{A<B}$ can be easily obtained inverting $B \leftrightarrow A$ in the previous table. The case $O_{A=B}$ is simple

EXAMPLE: COMPARATOR

```

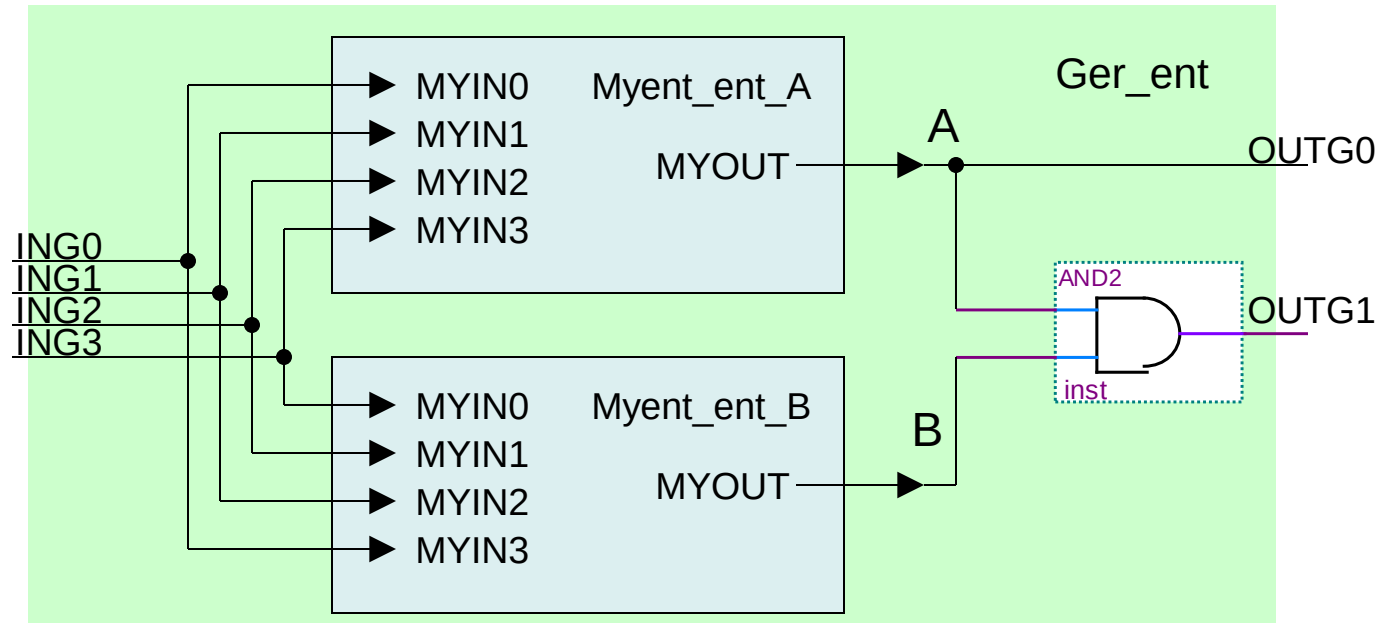
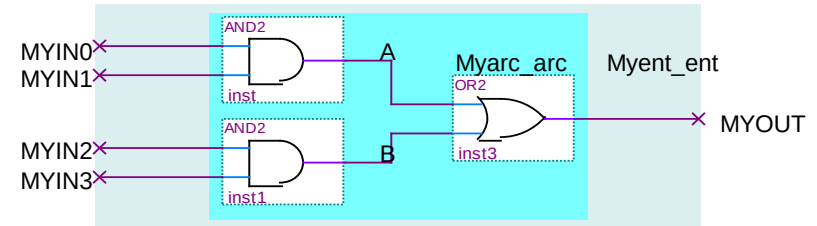
entity Comp_ent is
  port (
    A: in bit_vector(2 downto 0);
    B: in bit_vector(2 downto 0);
    Omag: out bit; --A>B
    Oeq: out bit;  --A=B
    Omin: out bit); --A<B
end Comp_ent ;
architecture Comp_arc of Comp_ent is
begin
  Comp: process (A,B)
  begin
    Omag <= '0'; Oeq <= '0'; Omin <= '0';
    if A=B then Oeq <= '1';
    elsif A(2 downto 1) = B(2 downto 1) then
      if A(0) = '1' and B(0) = '0' then Omag <= '1';
      elsif A(0) = '0' and B(0) = '1' then Omin <= '1';
      end if;
    elsif A(2) = B(2) then
      if A(1) = '1' and B(1) = '0' then Omag <= '1';
      elsif A(1) = '0' and B(1) = '1' then Omin <= '1';
      end if;
    else
      if A(2) = '1' and B(2) = '0' then Omag <= '1';
      elsif A(2) = '0' and B(2) = '1' then Omin <= '1';
      end if;
    end if;
  end process Comp;
end Comp_arc ;
  
```



ARCHITETTURA GERARCHICA

Una entity può includere altre entity per generare una struttura gerarchica.

Per esempio posso costruire l'entity 'Ger_ent' che include 2 'Myent_ent' e altra logica:



ARCHITETTURA GERARCHICA

```
entity Ger_ent is
  port ( ING0, ING1, ING2, ING3 : in bit;
        OUTG0, OUTG1: out bit );
end Ger_ent;
```

```
architecture Myarc_arc of Ger_ent is
```

```
  component Myent_ent
    port ( MYIN0, MYIN1, MYIN2, MYIN3 : in bit;
          MYOUT: out bit );
  end component;
```

```
  signal A, B: bit;
```

```
begin
```

```
  Myent_A: Myent_ent
```

```
    port map (MYIN0 =>ING0, MYIN1=> ING1, MYIN2=>ING2,
              MYIN3=>ING3, MYOUT=>A);
```

```
  Myent_B: Myent_ent
```

```
    port map (MYIN0 =>ING3, MYIN1=> ING2, MYIN2=>ING1,
              MYIN3=>ING0, MYOUT=>B);
```

```
  OUTG0 <= A;
```

```
  OUTG1 <= A and B;
```

```
end Myarc_arc ;
```

File 'Ger_ent.vhd'

Dichiarazione del 'sottoblocco' di tipo Myent_ent con la sua interfaccia – simile alla dichiarazione di un oggetto in C++

Implementazione di 2 oggetti di tipo Myent_ent e specifica delle connessioni

LIBRERIE

- E' possibile estendere il VHDL per mezzo di librerie
- Tali librerie possono essere costruite anche dagli utenti
- Nelle librerie si possono definire altri tipi di dati e operatori
- Particolarmente importante è la libreria `std_logic_1164` che è diventata uno standard di fatto.
- La/le librerie utilizzate si dichiarano nel file / entity che le utilizza così:

```
Library IEEE;  
use IEEE.std_logic_1164.all;  
  
entity Myent_ent is  
  port ( MYIN : in bit_vector (3 downto 0);  
        MYOUT: out bit );  
  
....
```

Dichiarazione
della libreria
`std_logic_1164`

STD_LOGIC

- La popolarità della libreria `std_logic_1164` sta nell'aggiunta del tipo di dato `'std_logic'` ed il corrispondente vettore `'std_logic_vector'`
- il dato di tipo `'std_logic'` estende il tipo `'bit'` e può assumere i valori:

Valore	Note	Uso tipico
'U'	Uninitialized	Usato in simulazione per la segnalazione di valori non inizializzati
'X'	Unknown	Usato in simulazione per la segnalazione di conflitti
'0'	0 logico	Valori già presenti nei 'bit'
'1'	1 logico	
'Z'	alta impedenza	Permette la gestione dei bus 3-states
'W'	weak unknown	Permettono la gestione di linee con pull up o pull down (per esempio comunicazioni standard I2C)
'L'	weak 0	
'H'	weak 1	
'_'	don't care	

EXAMPLE: COMPARATOR

Without Library

```
entity Comp_ent is
  port (  A: in bit_vector(2 downto 0);
         B: in bit_vector(2 downto 0);
         Omag: out bit, --A>B
         Oeq: out bit,  --A=B
         Omin: out bit); --A<B
end Comp_ent ;
architecture Comp_arc of Comp_ent is
begin
  Comp: process (A,B)
  begin
    Omag <= '0'; Oeq <= '0'; Omin <= '0';
    if A=B then Oeq <= '1';
    elsif A(2 downto 1) = B(2 downto 1) then
      if A(0) = '1' and B(0) = '0' then Omag <= '1';
      elsif A(0) = '0' and B(0) = '1' then Omin <= '1';
      end if;
    elsif A(2) = B(2) then
      if A(1) = '1' and B(1) = '0' then Omag <= '1';
      elsif A(1) = '0' and B(1) = '1' then Omin <= '1';
      end if;
    else
      if A(2) = '1' and B(2) = '0' then Omag <= '1';
      elsif A(2) = '0' and B(2) = '1' then Omin <= '1';
      end if;
    end if;
  end process Comp;
end Comp_arc ;
```

With Library

```
Library IEEE;
use IEEE.std_logic_1164.all;

entity Comp_ent is
  port (  A: in std_logic_vector(2 downto 0);
         B: in std_logic_vector (2 downto 0);
         Omag: out std_logic, --A>B
         Oeq: out std_logic,  --A=B
         Omin: out std_logic); --A<B
end Comp_ent ;
architecture Comp_arc of Comp_ent is
begin
  Comp: process (A,B)
  begin
    Omag = '0'; Oeq = '0'; Omin = '0';
    if A > B then Omag <= '1';
    elsif A = B then Oeq <= '1';
    else Omin <= '1';
    end if;
  end process Comp;
end Comp_arc ;
```

Among STD_Logic the symbols >, <, are defined, but not '+' or '-'

CIRCUITI COMBINATORI E SEQUENZIALI

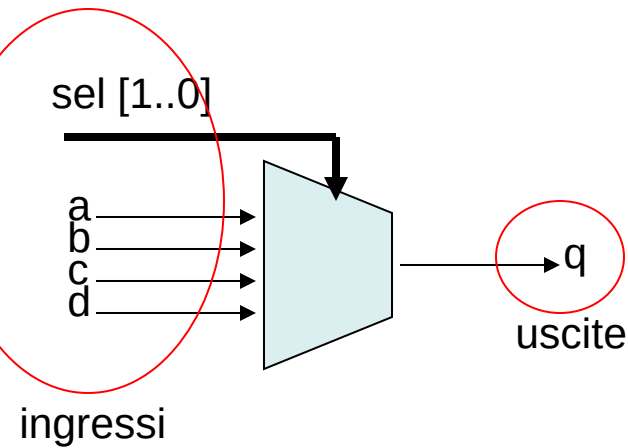
- Utilizzando le strutture del VHDL in teoria è possibile scrivere codici che realizzano funzioni molto 'strane'
- Anche limitandosi a usare istruzioni sintetizzabili (per esempio non usando 'wait' o 'real') non è detto che tutto ciò che si scrive sia sintetizzabile.
- Non è sufficiente seguire le regole sintattiche che abbiamo visto per scrivere codice che possa avere una corrispondenza hardware
- Inoltre le regole della "buona programmazione" impongono dei limiti ancora più ristretti su come effettivamente usare il codice VHDL

In pratica vi sono, al di là della sintassi VHDL, delle strutture e regole più o meno assodate per realizzare circuiti combinatori e sequenziali.

BLOCCHI COMBINATORI: REGOLE GENERALI

- Regola: la lista di sensibilità dei processi deve contenere TUTTI gli ingressi

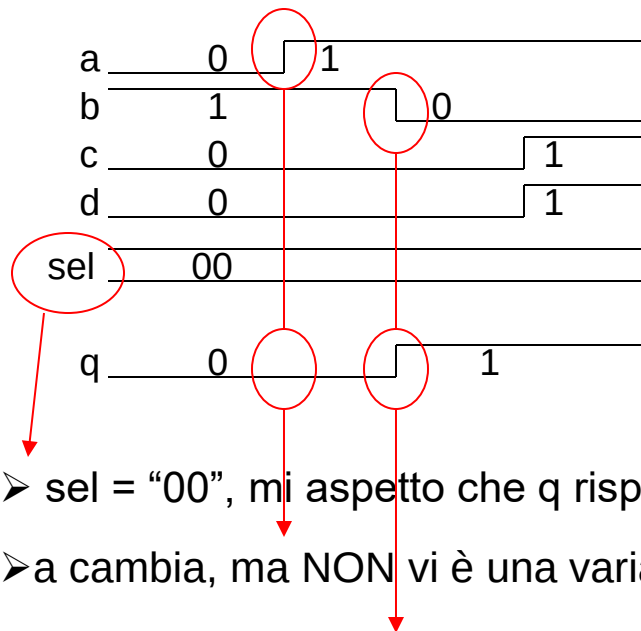
```
mux: process (a,b,c,d,sel)
begin
  case sel is
    when "00" =>
      q <= a;
    when "01" =>
      q <= b;
    when "10" =>
      q <= c;
    when others =>
      q <= d;
  end case;
end process mux;
```



BLOCCHI COMBINATORI: REGOLE GENERALI

➤ Esempio di errore: a omessa dalla sensitivity list

```
err: process (b,c,d,sel)
begin
  case sel is
    when "00" =>
      q <= a;
    when "01" =>
      q <= b;
    when "10" =>
      q <= c;
    when others =>
      q <= d;
  end case;
end process err;
```



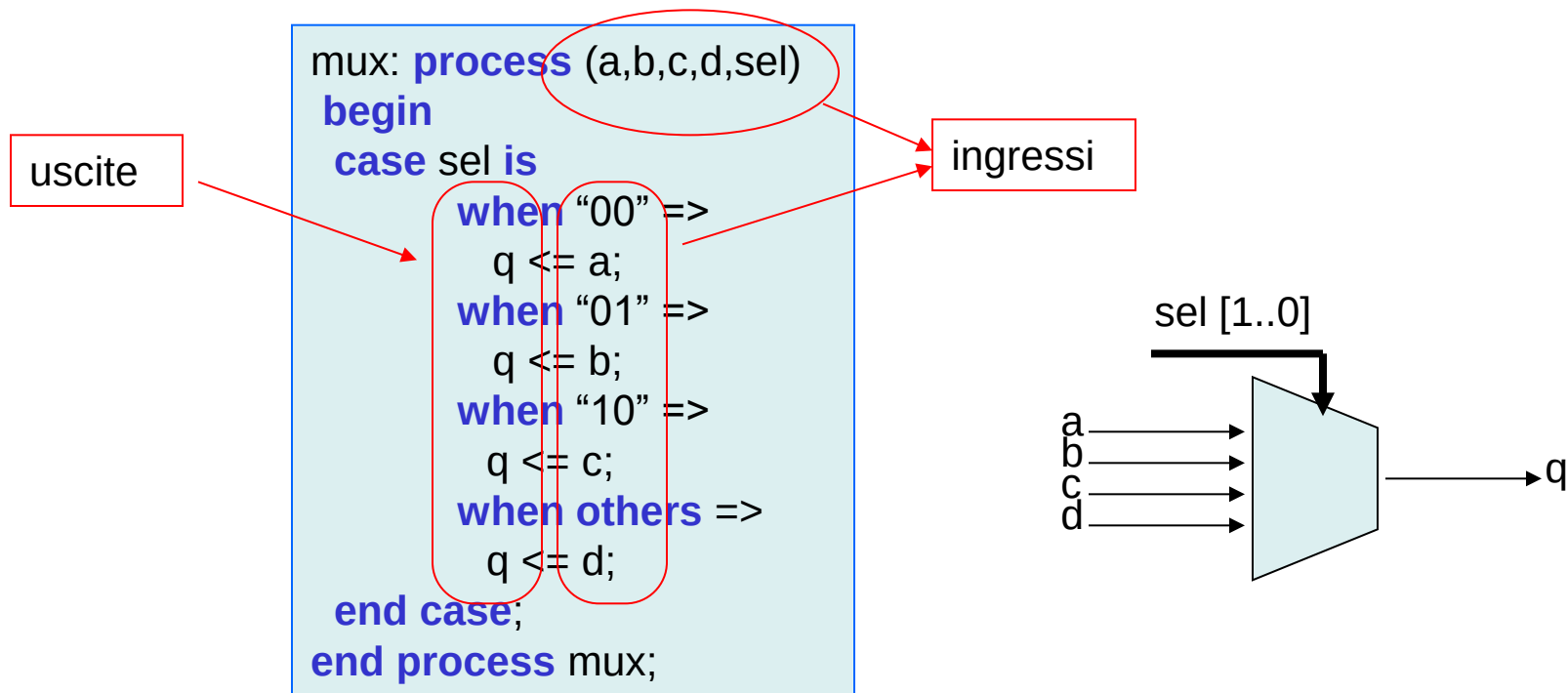
Poiché a non è nella sensitivity list, una sua variazione NON attiva il processo e q non è aggiornato. La variazione di b attiva il processo provocando l'aggiornamento di q

- sel = "00", mi aspetto che q rispecchi le variazioni di a
- a cambia, ma NON vi è una variazione di q
- Una variazione di b comporta l'aggiornamento di q

- L'uscita NON rispecchia immediatamente le variazioni dell'ingresso. Questo NON è un circuito combinatorio
- Il circuito è comunque sintetizzabile con l'uso di latch (elementi asincroni di memoria). In questi casi il sintetizzatore avvisa del potenziale errore.

BLOCCHI COMBINATORI: REGOLE GENERALI

➤ Regola: Netta distinzione fra ingressi e uscite

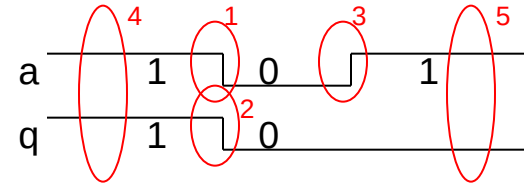
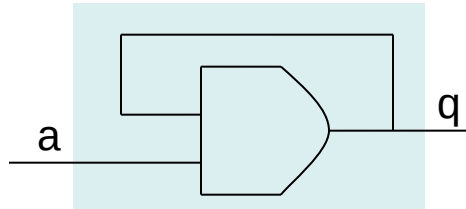


- Gli ingressi devono stare in sensitivity list e SOLO a destra delle assegnazioni (`<=`)
- Le uscite devono stare SOLO a sinistra delle assegnazioni (`<=`) e NON in sensitivity list

BLOCCHI COMBINATORI: REGOLE GENERALI

➤ Esempio di errore: uscita in ingresso e sensitivity list

```
err: process (a,q)
begin
    q <= a and q;
end process err;
```



➤ Sia inizialmente $q=1$ $a=1$. La transizione (1) provoca l'attivazione del processo al cui termine si aggiorna q generando la transizione (2). Essendo q in sensitivity list, questa provoca la riattivazione del processo che però non porta ad una nuova variazione di q . La successiva transizione (3) di a provoca l'attivazione del processo ma non ulteriori variazioni di q .

➤ Si notino le regioni (4) e (5). Ad ingresso '1' ($a=1$) si ottengono uscite diverse. L'uscita dipende dalla "storia" passata. Il circuito NON è puramente combinatorio

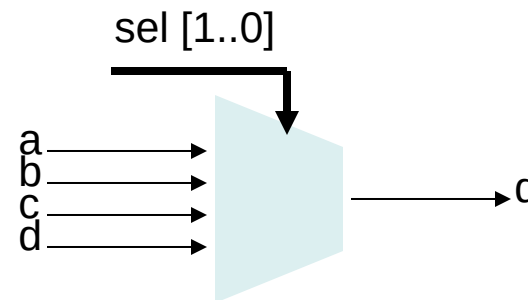
➤ La sintassi VHDL è corretta, il circuito può essere sintetizzato, ma non è né combinatorio né sequenziale sincrono: VA EVITATO per le regole della "buona programmazione"

BLOCCHI COMBINATORI: REGOLE GENERALI

➤ Regola: nelle assegnazioni condizionate (come in **if** e **case**) specificare il valore delle uscite per TUTTI i possibili valori della variabile di controllo. Questo è implicito se si usa **else** e **when others**.

```

mux: process (a,b,c,d,sel)
begin
  case sel is
    when "00" =>
      q <= a;
    when "01" =>
      q <= b;
    when "10" =>
      q <= c;
    when "11" =>
      q <= d;
  end case;
end process mux;
  
```



Il codice specifica quanto vale q per le tutte e 4 le combinazioni binarie che possono assumere i 2 bit di sel

BLOCCHI COMBINATORI: REGOLE GENERALI

➤ La regola precedente è implicitamente soddisfatta qualora si usi **else** e **when others**.

```
mux: process (a,b,c,d,sel)
begin
  case sel is
    when "00" =>
      q <= a;
    when "01" =>
      q <= b;
    when "10" =>
      q <= c;
    when others =>
      q <= d;
  end case;
end process mux;
```

```
mux: process (a,b,c,d,sel)
begin
  if sel = "00" then
    q <= a;
  elsif sel = "01" then
    q <= b;
  elsif sel = "10" then
    q <= c;
  else
    q <= d;
  end if;
end process mux;
```

```
architecture arc of Mux is
begin
  q <= a when sel = "00" else
    b when sel = "01" else
    c when sel = "10" else
    d;
end arc ;
```

```
architecture arc of Mux is
begin
  with sel select
    q <= a when "00",
    b when "01",
    c when "10",
    d when others;
end arc ;
```

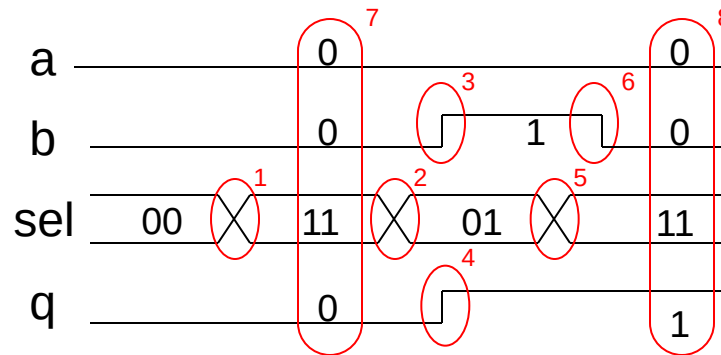
Sono automaticamente inclusi tutti i casi non esplicitamente elencati

BLOCCHI COMBINATORI: REGOLE GENERALI

➤ Esempio di errore: uscita non specificata per il caso sel = "11"

```

mux: process (a,b,c,d,sel)
begin
  if sel = "00" then
    q <= a;
  elsif sel= "01" then
    q <= b;
  elsif sel= "10" then
    q <= c;
  end if;
end process mux;
  
```



➤ Sia inizialmente sel="00" a='0' e q='0'. La transizione (1) provoca l'attivazione del processo, ma per sel = "11" non è previsto l'aggiornamento di q che

quindi mantiene il vecchio valore. (2) fa attivare il processo ma senza provocare variazioni. (3) provoca l'aggiornamento a '1' di q (4). (5) è analoga a (1), q rimane invariato. (6) non provoca variazioni.

❖ Si notino le regioni (7), (8) in cui, a fronte di ingressi identici si ha uscita diversa: il circuito NON è combinatorio